



Pending Agent Technical Documentation

Comprehensive guide to the Pending Agent — the autonomous reminder engine that manages caller follow-up cycles for on-hold incidents.

Incident AI Platform

Document Version 1.0 • October 2026

Generated from Source Code Analysis

Table of Contents

1. Overview & Purpose
2. Architecture & File Map
3. Core Concepts & Terminology
4. Trigger Mechanism — How the Pending Agent Starts
5. Complete Flow — Step by Step
6. Decision Paths — All Cases Handled
7. LLM Work Note Analyzer — Intelligence Layer
8. Reminder Cycle Lifecycle
9. Background Scheduler
10. Email Rendering & Templates
11. State Bounce Mechanism (on_hold ↔ in_progress)
12. Loop Prevention & Idempotency
13. Database Models & Schema
14. API Endpoints
15. Configuration & Tuning
16. Dashboard Metrics

1. Overview & Purpose

The **Pending Agent** is an autonomous AI-powered agent within the Incident AI Platform responsible for managing **caller follow-up reminder cycles** when an incident ticket is placed in `on_hold` / `pending` status.



Core Mission

When an engineer places a ticket on hold because they need information from the caller (the end user who raised the ticket), the Pending Agent automatically sends up to 3 escalating reminder emails to the caller at configurable intervals, tracks the cycle lifecycle, and logs everything as structured work notes.

What It Does

- **Detects** when an incident transitions from `active` / `in_progress` → `on_hold` / `pending`
- **Analyzes** the engineer's work note using an LLM (Anthropic Claude) to determine if the *caller* specifically needs to take action
- **Creates** a PendingCycle record to track the reminder sequence (up to 3 reminders)
- **Schedules** reminders at configurable intervals (default: 20 seconds in dev, 3600 seconds in production)
- **Renders** professional escalating reminder emails (Reminder 1 → 2 → Final Warning)
- **Logs** every action as a structured `IncidentWorkNote` with full audit trail
- **Manages state** — bounces incident state from `on_hold` → `in_progress` → `on_hold` to show activity
- **Completes** the cycle after all 3 reminders are exhausted

What It Does NOT Do

- Does **not** resolve or close tickets — that is the Resolution Agent's job
- Does **not** send reminders for internal waits (approvals, escalations, vendor dependencies)
- Does **not** create cycles when an engineer sets `on_hold` without a substantive user-directed work note

2. Architecture & File Map

The Pending Agent follows a clean layered architecture: **Event → Handler → Service → Agent + Analyzer + Renderer → Repository → Database.**

Complete File Inventory

Core Pending Agent Files (7 files)

FILE	ROLE	LINES
modules/agents/pending/agent.py	PendingAgent class — pure reasoning (extends BaseAgent)	62
modules/agents/pending/service.py	PendingService — main orchestrator (662 lines, the heart of the agent)	662
modules/agents/pending/work_note_analyzer.py	PendingWorkNoteAnalyzer — LLM-powered note analysis	93
modules/agents/pending/reminder_renderer.py	PendingReminderRenderer — email text & HTML generation	85
modules/agents/pending/scheduler.py	Background reminder polling loop	68
modules/agents/pending/repository.py	PendingCycleRepository (agent-level, legacy)	103
modules/agents/pending/schemas.py	PendingWorkNoteAnalysis & PendingResult Pydantic models	32

Supporting Models & Templates (3 files)

FILE	ROLE
modules/agents/pending/models/pending_cycle.py	SQLAlchemy PendingCycle model (agent-level definition)

FILE	ROLE
<code>modules/agents/pending/templates/standard_pending_reminder.html</code>	Jinja2 HTML email template with escalating urgency styling
<code>modules/agents/pending/models/__init__.py</code>	Model exports

Shared Pending Cycle Module (6 files)

FILE	ROLE
<code>modules/pending_cycles/model.py</code>	Canonical PendingCycle SQLAlchemy model with partial unique index
<code>modules/pending_cycles/enums.py</code>	PendingCycleStatus enum (ACTIVE, COMPLETED, CANCELLED)
<code>modules/pending_cycles/repository.py</code>	PendingCycleRepository — all DB queries for pending_cycles table
<code>modules/pending_cycles/service.py</code>	PendingCycleService — business logic, idempotent operations
<code>modules/pending_cycles/schemas.py</code>	Pydantic schemas (Create, Update, Response, ListResponse)
<code>modules/pending_cycles/__init__.py</code>	Module exports

Orchestration & API Layer (4 files)

FILE	ROLE
<code>orchestrator/handlers/pending.py</code>	PendingHandler — event listener for IncidentStateChangedEvent
<code>orchestrator/events.py</code>	IncidentStateChangedEvent dataclass definition
<code>api/v1/pending.py</code>	REST API endpoints (POST trigger, GET cycle status)
<code>core/lifecycle.py</code>	App startup — launches background scheduler

Shared Infrastructure (used by Pending Agent)

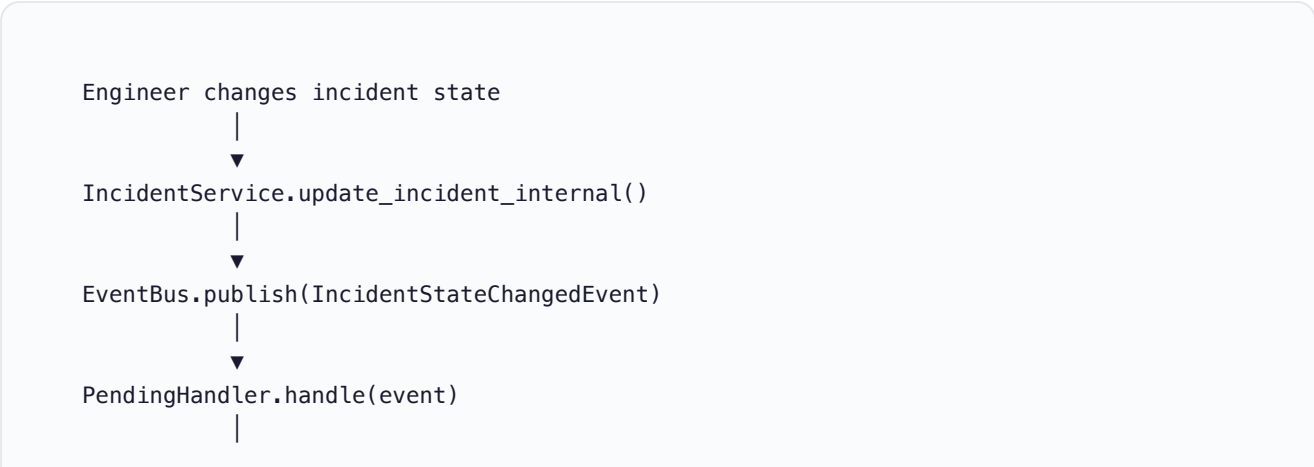
FILE	ROLE
<code>modules/agents/base/base_agent.py</code>	Abstract BaseAgent class
<code>modules/agents/base/response.py</code>	AgentResponse Pydantic model
<code>ai_platform/services/llm_service.py</code>	LLMService — Anthropic Claude API wrapper
<code>orchestrator/bus.py</code>	In-process async EventBus
<code>orchestrator/constants.py</code>	AgentSource canonical constants
<code>modules/work_notes/enums.py</code>	WorkNoteSourceType & WorkNoteActionType enums
<code>modules/work_notes/service.py</code>	WorkNoteService — add_note(), get_notes()
<code>modules/incidents/service.py</code>	IncidentService — get/update incident

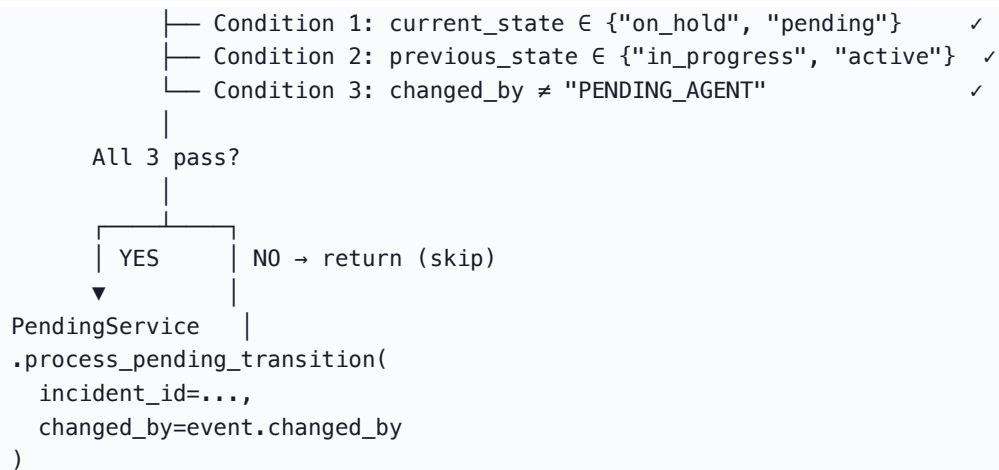
3. Core Concepts & Terminology

TERM	DEFINITION
PendingCycle	A first-class database entity that tracks a sequence of up to 3 reminders for one incident. An incident can have multiple sequential cycles over its lifetime, but at most one ACTIVE cycle at any given time (enforced by a partial unique index).
Reminder Tier	The ordinal position within a cycle — Reminder 1, Reminder 2, or Reminder 3 (the final warning). Each tier has escalating urgency in tone and email design.
Caller	The end user who raised the incident ticket. This is the person reminders are sent to.
State Bounce	The intentional pattern where the Pending Agent briefly moves the incident from <code>on_hold</code> → <code>in_progress</code> (to show activity) and then immediately restores it to <code>on_hold</code> .
Work Note Analysis	LLM-powered classification of the engineer's work note to determine if the <i>caller</i> specifically needs to take action (provide info, confirm, test something) vs. internal waits (approvals, escalations, vendor).
force_reminder	A boolean flag. When <code>False</code> , the service creates/preserves a cycle. When <code>True</code> , the service executes the next due reminder (called by the scheduler).

4. Trigger Mechanism — How the Pending Agent Starts

The Pending Agent is triggered through the **EventBus** when an incident's state changes. Here is the exact triggering chain:





Key Condition: Previous State Must Be Active

The `previous_state` must be `"in_progress"` or `"active"`. This prevents triggering on:

- `new → on_hold` (not preceded by active state)
- `on_hold → on_hold` (redundant write, no actual transition)
- The Pending Agent's own restore of `on_hold` (uses `changed_by=PENDING_AGENT`)

5. Complete Flow — Step by Step

Phase 1: Initial Trigger (Cycle Creation)

1. Engineer adds a work note like *"Caller, please provide the error screenshot"* and sets the incident to `on_hold`.
2. `IncidentService` persists the state change and publishes `IncidentStateChangedEvent`.
3. `PendingHandler` receives the event, validates all 3 conditions, and calls `PendingService.process_pending_transition()`.
4. `PendingService` checks for an existing active cycle → none found.
5. If `changed_by == "ENGINEER"`: runs LLM analysis on the engineer's work note to determine if the *caller* needs to act. If the LLM says "no, this is an internal wait" → **skips cycle creation**.
6. If analysis says caller action IS required → creates a new `PendingCycle` via `PendingCycleService.create_cycle_if_not_exists()`.
7. Increments `reminder_count` from 0 → 1 and sets `next_reminder_at = now + REMINDER_INTERVAL`.
8. Ensures incident is in `on_hold` state.
9. Returns `AgentResponse` with `action_taken="REMINDER_SCHEDULED"`.

Phase 2: Reminder Execution (Scheduler Triggers)

1. Background `start_pending_reminder_scheduler()` polls every 10 seconds for due cycles.
2. Finds cycles where `status=ACTIVE` and `next_reminder_at ≤ now`.
3. Calls `PendingService.process_pending_transition(force_reminder=True)`.
4. Service fetches latest 15 work notes, runs LLM analysis for context.
5. Renders the reminder email (plain text for work note, HTML available for email).
6. Adds a `PENDING_AGENT / SEND_REMINDER` work note with `auto_activate=True`.
7. This causes the incident to bounce: `on_hold → in_progress` (showing activity).
8. Immediately restores `on_hold` via `IncidentService` with `changed_by="PENDING_AGENT"`.
9. Adds a second work note recording the state restoration.
10. If not the final reminder: increments `reminder_count` and schedules next reminder.
11. If final reminder (3/3): marks cycle as `COMPLETED`, clears `next_reminder_at`.

Phase 3: Cycle Completion

1. After Reminder 3 is sent, the cycle status becomes `COMPLETED`.
2. `next_reminder_at` is cleared → scheduler no longer picks up this cycle.
3. Incident remains in `on_hold` state — manual engineer follow-up is required.
4. A new cycle can be created if the engineer transitions the ticket again.

6. Decision Paths — All Cases Handled

The `PendingService.process_pending_transition()` method handles **6 distinct cases**:

PATH	CONDITION	ACTION	RESULT
Path A	Active cycle exists + <code>force_reminder=False</code>	Preserve existing cycle (idempotency gate)	<code>CYCLE_PRESERVED</code>
Path A2	No cycle + <code>changed_by="ENGINEER"</code> + LLM says no caller action needed	Skip cycle creation	<code>SKIPPED_NO_USER_ACTION_REQUIRED</code>
Path B	No active cycle + <code>force_reminder=False</code>	Create cycle, schedule Reminder 1	<code>REMINDER_SCHEDULED</code>
Path C	<code>force_reminder=True</code> + active cycle exists + <code>reminder_count < max</code>	Execute reminder, schedule next	<code>REMINDER_SENT</code>
Path C (final)	<code>force_reminder=True</code> + <code>reminder_count = max_reminders</code>	Execute final reminder, complete cycle	<code>CYCLE_COMPLETED</code>
Path C (guard)	<code>force_reminder=True</code> + no active cycle (cancelled/completed between scheduler query and execution)	Skip safely	<code>DISCARDED</code>

✅ Over-Limit Guard

If `reminder_count > max_reminders` when execution arrives (stale scheduler job or duplicate call), the cycle is silently completed without sending another reminder. This prevents over-sending.

7. LLM Work Note Analyzer — Intelligence Layer

Purpose

The `PendingWorkNoteAnalyzer` uses Anthropic Claude to classify whether the **caller (end user)** needs to take action based on the engineer's work note content.

File

`modules/agents/pending/work_note_analyzer.py` — 93 lines

Classification Logic

RESULT	WHEN	EXAMPLES
<code>is_caller_action_required = true</code>	Engineer explicitly asks the caller to do something	"Please provide error screenshot" "Confirm if issue is resolved" "Try restarting your machine" "Provide your employee ID"
<code>is_caller_action_required = false</code>	Engineer is waiting for internal action	"Escalating to infrastructure team" "Waiting for manager approval" "Pending vendor response" "Change window scheduled for next week"

LLM System Prompt (Key Excerpt)

```
// The LLM receives this system prompt:
You are an expert ITSM classifier for ServiceNow incidents.
An engineer has added a work note and placed it in ON HOLD.
Determine whether the CALLER needs to take action.

Return ONLY valid JSON:
{
  "is_caller_action_required": true or false,
  "reasoning": "One sentence explaining...",
  "confidence": float 0.0 to 1.0
}
```

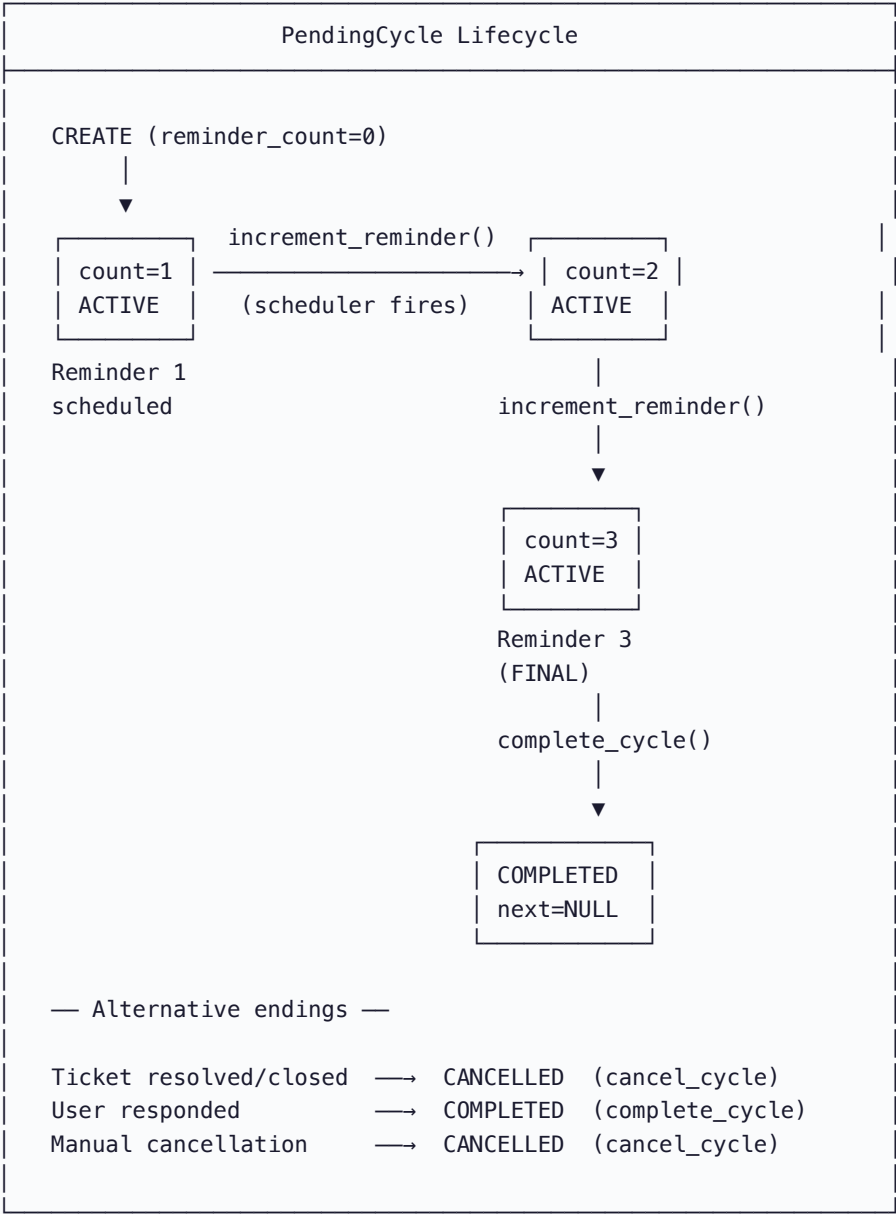
Fast Path

If the work note source is `AcknowledgementAgent`, the analyzer bypasses the LLM call entirely and returns `is_caller_action_required=true` with confidence `1.0` — since acknowledgement flow always requires user action (RITM submission).

Fallback

If the LLM call fails (network error, JSON parse error, etc.), the analyzer defaults to `is_caller_action_required=true` with confidence `0.8`. This is a **safe default** — it's better to send an unnecessary reminder than to miss one.

8. Reminder Cycle Lifecycle



Cycle Status Transitions

STATUS	MEANING	NEXT_REMINDER_AT
ACTIVE	Cycle is running, reminders are being sent	Set to future timestamp

STATUS	MEANING	NEXT_REMINDER_AT
COMPLETED	All reminders sent, or caller responded, cycle ended normally	NULL
CANCELLED	Ticket moved out of pending, or manually aborted	NULL

Single Active Cycle Constraint

The database enforces a **partial unique index** on `(incident_id)` where `status = 'ACTIVE'`. This means at most one ACTIVE cycle can exist per incident at any time. Concurrent insert attempts are caught by `IntegrityError` and handled gracefully.

9. Background Scheduler

File

`modules/agents/pending/scheduler.py`

How It Works

1. Launched at app startup in `core/lifecycle.py` as an `asyncio.create_task()`.
2. Runs an infinite `while True` loop, sleeping for `poll_interval_seconds` (default 10s) between iterations.
3. Each iteration queries `PendingCycleRepository.get_due_active_cycles()` — finds all ACTIVE cycles where `next_reminder_at ≤ now`.
4. For each due cycle, creates a new `PendingService` with a fresh DB session and calls `process_pending_transition(force_reminder=True)`.
5. On `CancelledError` (app shutdown), breaks cleanly.
6. On any other error, logs the error and continues (never crashes the scheduler loop).

Drift Prevention

Reminder scheduling uses
anchor-based timing

: the next reminder is scheduled relative to the *previous scheduled time*, not the wall-clock time after execution. This prevents interval drift caused by LLM/email execution time.

Example: If Reminder 1 was scheduled at $T+20s$, Reminder 2 fires at $T+40s$ — regardless of how long Reminder 1's execution took.

10. Email Rendering & Templates

Two Rendering Modes

MODE	METHOD	USAGE
Plain Text	<code>render_plain_text_email(context)</code>	Stored inside work notes for audit trail
HTML	<code>render_html(context)</code>	Jinja2 template for actual email delivery

Escalating Tone by Reminder Tier

TIER	SUBJECT	STONE	HEADER COLOR
Reminder 1	"Follow-up Reminder (1/3) - Ticket INC..."	Polite request: "Additional Information Needed"	Amber (#d97706)
Reminder 2	"Follow-up Reminder (2/3) - Awaiting Response..."	Firm follow-up: "Information Needed"	Amber (#d97706)
Reminder 3 (Final)	"FINAL REMINDER (3/3) - Action Required..."	Urgent: "Action Required to Prevent Ticket Closure"	Red (#b91c1c)

Template Context Variables

```
{
  "ticket_number":      "INC0000042",
  "caller_name":        "John Smith",
  "short_description":   "Cannot access VPN",
  "assignment_group":    "Network Support",
  "assigned_engineer_name": "Priya Sharma",
  "reminder_count":      2,
  "max_reminders":       3,
  "is_final":            false,
  "awaiting_detail":     "Please provide the error screenshot..."
}
```

11. State Bounce Mechanism (on_hold ↔ in_progress)

When a reminder is executed, the Pending Agent performs an intentional **"state bounce"**:

```
INCIDENT STATE: on_hold
|
Step 1: WorkNoteService.add_note(auto_activate=True)
|       → adds SEND_REMINDER work note
|       → auto_activate triggers: on_hold → in_progress
|
INCIDENT STATE: in_progress ← shows activity on the ticket
|
Step 2: IncidentService.update_incident_internal(
|       state="on_hold",
|       changed_by="PENDING_AGENT" ← critical: prevents re-trigger
|   )
|
INCIDENT STATE: on_hold ← restored, awaiting caller
```

⚠ Critical: changed_by="PENDING_AGENT"

The restore to `on_hold` uses `changed_by="PENDING_AGENT"`. The `PendingHandler` has an explicit guard: `if event.changed_by == AgentSource.PENDING_AGENT: return`. Without this, the state bounce would trigger a new cycle creation → infinite loop.

12. Loop Prevention & Idempotency

Three Layers of Protection

LAYER	MECHANISM	WHAT IT PREVENTS
1. Handler Filter	<code>PendingHandler</code> skips events where <code>changed_by == "PENDING_AGENT"</code>	Infinite loop from state bounce (on_hold → in_progress → on_hold)
2. Previous State Check	<code>PendingHandler</code> requires <code>previous_state ∈ {"in_progress", "active"}</code>	Duplicate triggers from on_hold → on_hold writes or new → on_hold
3. Idempotent Cycle Creation	<code>PendingService</code> checks for existing active cycle before creating new one	Duplicate cycles from concurrent requests

Database-Level Constraint

The `pending_cycles` table has a **partial unique index**:

```
CREATE UNIQUE INDEX uq_active_cycle_per_incident
ON pending_cycles (incident_id)
WHERE status = 'ACTIVE';
```

This guarantees at most one ACTIVE cycle per incident, even under concurrent inserts. The `IntegrityError` is caught and resolved by re-fetching the existing cycle.

13. Database Models & Schema

pending_cycles Table

COLUMN	TYPE	NULLABLE	DESCRIPTION
<code>id</code>	String(36)	PK	UUID primary key
<code>incident_id</code>	String(36)	FK	Foreign key → incidents.id (CASCADE delete)

COLUMN	TYPE	NULLABLE	DESCRIPTION
<code>status</code>	String(20)	NOT NULL	ACTIVE, COMPLETED, or CANCELLED
<code>reminder_count</code>	Integer	NOT NULL	Current reminder count (0 on creation, incremented to 1, 2, 3)
<code>max_reminders</code>	Integer	NOT NULL	Maximum reminders (default 3)
<code>reminder_template</code>	String(100)	YES	Template identifier (reserved for future use)
<code>next_reminder_at</code>	DateTime(tz)	YES	When the next reminder should fire; NULL when cycle is done
<code>created_at</code>	DateTime(tz)	NOT NULL	Cycle creation timestamp (UTC)
<code>updated_at</code>	DateTime(tz)	NOT NULL	Last update timestamp (UTC)
<code>completed_at</code>	DateTime(tz)	YES	When cycle completed (NULL until COMPLETED)
<code>cancelled_at</code>	DateTime(tz)	YES	When cycle cancelled (NULL until CANCELLED)

Indexes

INDEX NAME	COLUMNS	TYPE
<code>ix_pending_cycles_incident_id</code>	incident_id	B-tree
<code>ix_pending_cycles_status</code>	status	B-tree
<code>uq_active_cycle_per_incident</code>	incident_id (WHERE status='ACTIVE')	Partial Unique

Work Notes Created by Pending Agent

Each reminder execution creates **2 work notes**:

#	SOURCE_TYPE	ACTION_TYPE	CONTENT
1	PENDING_AGENT	SEND_REMINDER	Reminder details + full email text + cycle status
2	PENDING_AGENT	STATE_CHANGE	"State restored to On Hold after reminder N sent"

14. API Endpoints

POST /v1/pending/{incident_id}

Trigger Pending Agent

Manually trigger the Pending Agent for an incident. Typically called automatically via the EventBus, but can be invoked manually for testing.

PARAMETER	TYPE	REQUIRED	DESCRIPTION
<code>incident_id</code>	path	Yes	UUID of the incident
<code>force_reminder</code>	query	No (default: false)	If true, advances reminder count even if cycle is active

Response: `AgentResponse` JSON with `success`, `agent_name`, `reasoning`, `confidence`, and `result` (PendingResult object).

GET /v1/pending/{incident_id}/cycle

Get Cycle Status

Returns the active or latest pending cycle for an incident.

Response Example:

```
{
  "status": "success",
  "cycle": {
    "id": "a1b2c3d4-...",
    "incident_id": "e5f6g7h8-...",
    "status": "ACTIVE",
    "reminder_count": 2,
    "max_reminders": 3,
    "next_reminder_at": "2026-10-01T12:00:00+00:00",
    "created_at": "2026-10-01T11:59:20+00:00",
    "updated_at": "2026-10-01T11:59:40+00:00"
  }
}
```

```
}  
}
```

15. Configuration & Tuning

PARAMETER	LOCATION	DEFAULT	DESCRIPTION
<code>REMINDER_INTERVAL_SECONDS</code>	<code>service.py</code> (line 54)	20 (dev)	Delay between reminder tiers. Set to 3600 (1 hour) for production.
<code>poll_interval_seconds</code>	<code>lifecycle.py</code> (line 20)	10	How often the scheduler checks for due reminders.
<code>max_reminders</code>	<code>PendingCycleService</code>	3	Maximum reminders per cycle. Configurable per cycle creation.
<code>ANTHROPIC_MODEL</code>	<code>.env / settings</code>	Configured in app	LLM model used for work note analysis.

16. Dashboard Metrics

The Pending Agent feeds several dashboard components via the `/v1/dashboard/allstats` API:

DASHBOARD CARD	WHAT IT SHOWS	DATA SOURCE
Active Pending Cycles	Count of incidents currently in active reminder cycles	Incidents with <code>PENDING_AGENT</code> work notes + on_hold count
Pending Cycle Status Donut	Breakdown: Active vs Completed vs Cancelled cycles	Derived from pending_cnt, resolved_cnt, on_hold counts
Reminder Distribution Bar	How many Reminder 1s, 2s, and 3s were sent	Counted from <code>SEND_REMINDER</code> work notes matching "reminder 1/2/3" in message text

DASHBOARD CARD	WHAT IT SHOWS	DATA SOURCE
Agent Activity 24h Chart	Pending Agent activity over time (hourly or daily)	Counted from <code>PENDING_AGENT</code> source_type work notes
Incident Automation Funnel	"Pending Workflow" stage — distinct incidents that entered pending	Count of distinct incidents with <code>PENDING_AGENT</code> work notes
Response Rate (R1, R2, R3)	At which reminder tier users respond	Strict timestamp sequence: user response between Reminder N and N+1 timestamps
Recent Agent Activity	Shows "Pending" agent rows with actions like "Reminder 1/2/3 sent"	Latest <code>PENDING_AGENT</code> work notes per incident